



Saga Pattern: Managing Distributed Transactions

Solving Consistency in Microservices

2.

Agenda



1. Challenges and Introduction to Sagas
2. Saga Flow and Coordination Models
3. Compensation and Reliability
4. Implementation and Key Considerations

3.



Challenges and Introduction to Sagas

4.



Distributed Transactions Challenge



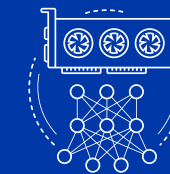
Microservices span multiple databases

Each service manages its own data store



No ACID guarantees across services

Atomicity and consistency are not enforced globally



Traditional transactions can't cross service boundaries

Distributed transaction problem emerges

5.



Why 2-Phase Commit Falls Short



Service Blocking Limits Performance

2-phase commit forces services to wait, reducing throughput.



Tight System Coupling

Strong dependencies make systems harder to evolve independently.



Vulnerable to Distributed Failures

Network or node issues can halt the entire transaction process.

6.



Saga: A Smarter Approach

Decomposes Transactions

Splits complex operations into manageable local steps.

Event-Driven Coordination

Services communicate through events for workflow progression.

Ensures Eventual Consistency

System reaches a consistent state over time, not instantly.

7.



Saga Flow and Coordination Models

8.



How Saga Flows Work



9.



Choreography-Based Sagas

Services operate independently

No central controller manages the workflow

Event-driven progression

Each service listens and reacts to relevant events

Autonomous step triggering

Services initiate the next action after completing their own

10.



Choreography: Pros and Cons



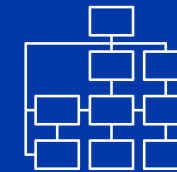
Loose Coupling Enhances Flexibility

Services operate independently, reducing dependencies.



Scalable for Growing Systems

Easily adapts to increased load and new services.



Complexity in Workflow Visualization

Difficult to track and manage intricate event flows.

11.



Orchestration-Based Sagas



Centralized Saga Coordination

A single orchestrator manages the entire transaction flow.



Explicit Command Issuance

Orchestrator sends commands to each service for every step.



State Tracking and Recovery

Tracks progress and handles failures or compensations.



Orchestration: Pros and Cons

Aspect	Orchestration-Based Saga
Visibility	Clear workflow tracking
Coordination	Central controller required
Coupling	Central dependency introduced
Complexity	Simplifies understanding

13.



Compensation and Reliability



Compensation Transactions

Business Logic for Undoing Actions

Compensations reverse previous steps using domain-specific rules.

Semantic Reversal, Not Rollback

Actions are undone meaningfully, not just technically.

Tailored to Each Service

Compensation steps depend on business context and requirements.

15.



Designing Reliable Compensations



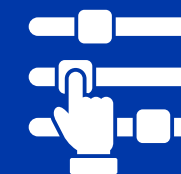
Idempotent Compensation Logic

Ensures repeated actions do not cause unintended effects.



Safe Retry Mechanisms

Handles retries without data corruption or duplication.



Graceful Handling of Partial Failures

Compensations recover from incomplete or failed steps.

16.



Implementation and Key Considerations



Saga Implementation Choices

Event-Driven Saga Execution

Leverage messaging systems to coordinate saga steps.

Workflow Engines for Automation

Automate saga flows with workflow management tools.

Custom Orchestrators

Build tailored controllers for complex saga requirements.



Handling Saga Failures

Failure Scenario	Reliability Strategy
Partial Failure	Compensation transactions
Duplicate Events	Idempotent operations
Retry Logic	Safe to retry compensations
Event Loss	Robust messaging systems

19.



Key Takeaways



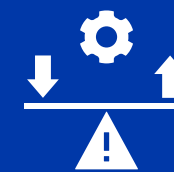
Distributed ACID Is Unachievable

Microservices must rely on eventual consistency.



Sagas Enable Reliable Microservices

Break complex transactions into manageable steps.



Compensation Logic Is Critical

Design compensations to be idempotent and safe to retry.